

Dockerfile.hardened

Enterprise DevSecOps Security Lab

Professional Container Hardening Design, Implementation, Validation, and Evidence Guide

Field	Value
Author	Jagriti Banerjee
Document Type	Container Security Hardening Document
Project	Enterprise DevSecOps Security Lab
Artifact	Dockerfile.hardened
Scope	Flask application container image, local Docker, CI/CD scanning, Kubernetes deployment compatibility
Primary Security Themes	Non-root execution, multi-stage build, minimal runtime, healthcheck, immutable application code, SBOM/signing readiness
Environment	Private VM / Docker / Kind Kubernetes cluster / GitHub Actions

Purpose: This document provides an enterprise-grade hardened Dockerfile design for the Flask application used in the Enterprise DevSecOps Security Lab. It explains the security controls, implementation decisions, validation commands, expected evidence, risk considerations, and CI/CD integration points.

Important: The vulnerable Flask routes are intentionally retained for lab demonstration. The Dockerfile hardening reduces container and runtime risk, but it does not replace secure code remediation for SQL injection, command injection, unsafe rendering, or dependency vulnerabilities.

Security Outcome	Target State
Container identity	Runs as fixed non-root UID/GID 10001:10001
Build hygiene	Uses multi-stage build to keep dependency installation separate from runtime
Runtime exposure	Only exposes application port 5000; public exposure should remain disabled in private lab
Runtime server	Uses Gunicorn instead of Flask development server
Evidence readiness	Compatible with Trivy, Syft SBOM, Cosign signing, GitHub Actions security gates

1. Executive Summary

Dockerfile.hardened is designed as the secure container build artifact for the Enterprise DevSecOps Security Lab. It converts the Flask application from a basic lab container into a production-style runtime image with stronger security defaults, repeatable build behavior, scanner compatibility, and Kubernetes SecurityContext alignment.

The hardened Dockerfile improves the project by showing that the container lifecycle is not treated as an afterthought. Instead, container security is built into the artifact itself, then validated through Docker runtime checks, Trivy image scanning, Syft SBOM generation, Cosign signature verification, and Kubernetes deployment hardening.

This document is intended to be placed in the project documentation set and used alongside Dockerfile, docker-compose.yml, security-gates.yml, supply chain reports, Kubernetes assessment reports, and retest evidence.

1.1 Key Security Outcomes

- Build dependencies are separated from runtime dependencies using a multi-stage Docker build.
- The container runs as a dedicated non-root user with a fixed UID and GID.
- Application code permissions are hardened to reduce runtime tampering risk.
- Gunicorn is used as the runtime server instead of the Flask development server.
- A Docker HEALTHCHECK validates application availability through the /health endpoint.
- The image is compatible with Kubernetes runAsNonRoot, readOnlyRootFilesystem, dropped capabilities, and RuntimeDefault seccomp settings.
- The image can be scanned by Trivy, inventoried by Syft, signed by Cosign, and included in CI/CD security gates.

2. Hardened Dockerfile Artifact

The following Dockerfile.hardened can be copied into the repository root as Dockerfile.hardened. It should be used as a secure baseline. If the application writes SQLite files or other runtime state, configure the application to write to /tmp/app or mount a dedicated writable volume in Kubernetes.

```
# syntax=docker/dockerfile:1.7

# -----
# Enterprise DevSecOps Security Lab - Hardened Dockerfile
# Maintainer: Jagriti Banerjee
# Purpose: Production-style hardened container build for the Flask demo app.
# -----

ARG PYTHON_VERSION=3.11.9

# -----
# Stage 1: Dependency builder
# - Keeps build tooling and package installation separate from runtime image.
# - Creates a virtual environment that is copied into the runtime stage.
# -----
FROM python:${PYTHON_VERSION}-slim-bookworm AS builder

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /build

COPY src/app/requirements.txt ./requirements.txt

RUN python -m venv /opt/venv \
    && /opt/venv/bin/pip install --upgrade pip setuptools wheel \
    && /opt/venv/bin/pip install --requirement requirements.txt

# -----
# Stage 2: Runtime image
# - Minimal runtime image.
```

```
# - Non-root user.
# - No build cache.
# - Healthcheck enabled.
# - Gunicorn used instead of Flask debug server.
# -----
FROM python:${PYTHON_VERSION}-slim-bookworm AS runtime

LABEL org.opencontainers.image.title="Enterprise DevSecOps Flask App" \
      org.opencontainers.image.description="Hardened runtime image for private DevSecOps
      security lab" \
      org.opencontainers.image.authors="Jagriti Banerjee" \
org.opencontainers.image.source="https://github.com/YOUR_GITHUB_USERNAME/Enterprise-DevSecOps"
\
      org.opencontainers.image.licenses="MIT"

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    PATH="/opt/venv/bin:${PATH}" \
    FLASK_ENV=production \
    APP_HOME=/app \
    APP_TMP=/tmp/app

WORKDIR ${APP_HOME}

# Create a fixed non-root identity for predictable Kubernetes SecurityContext use.
RUN addgroup --system --gid 10001 appgroup \
    && adduser --system --uid 10001 --ingroup appgroup --home ${APP_HOME} --shell
    /usr/sbin/nologin appuser \
    && mkdir -p ${APP_HOME} ${APP_TMP} \
    && chown -R appuser:appgroup ${APP_HOME} ${APP_TMP}

# Copy only runtime dependencies and application source.
COPY --from=builder /opt/venv /opt/venv
COPY --chown=appuser:appgroup src/app/ ${APP_HOME}/

# Harden file permissions. Application code should not be modified at runtime.
RUN find ${APP_HOME} -type f -exec chmod 0444 {} \; \
    && find ${APP_HOME} -type d -exec chmod 0555 {} \; \
    && chmod 1777 ${APP_TMP}

USER 10001:10001

EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
    CMD python -c "import urllib.request;
    urllib.request.urlopen('http://127.0.0.1:5000/health', timeout=3).read()" || exit 1

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--threads", "2", "--timeout",
    "30", "--access-logfile", "-", "--error-logfile", "-", "app:app"]
```

3. Why This Dockerfile Is Enterprise-Grade

A professional Dockerfile should not only build an image; it should reduce attack surface, improve runtime isolation, support security scanning, and create evidence that can be defended in a security review. The hardened Dockerfile implements the following enterprise-style design principles.

Control	Implementation	Security Value
Multi-stage build	Dependency installation occurs in builder stage; runtime stage receives only /opt/venv and app code.	Reduces runtime image complexity and prevents build-time artifacts from becoming runtime attack surface.
Minimal base image	Uses python:-slim-bookworm.	Reduces package footprint compared with full distributions while preserving compatibility for Python runtime.
Non-root user	Creates appuser with UID/GID 10001 and switches to USER 10001:10001.	Reduces impact if the application process is compromised. Aligns with Kubernetes runAsNonRoot.
No Flask debug server	Uses Gunicorn as the default CMD.	Avoids development server behavior and supports production-style worker management.
Healthcheck	Calls /health using Python urllib.	Allows Docker and orchestration layers to detect unhealthy application behavior.

Control	Implementation	Security Value
File permissions	Application files set to read-only permissions; /tmp/app made writable.	Reduces runtime tampering risk while preserving a safe temporary write location.
OCI labels	Adds title, description, author, source, and license metadata.	Improves artifact traceability and supply-chain documentation.
Scanner readiness	Compatible with Trivy, Syft, Cosign, and CI/CD security evidence workflows.	Supports enterprise-style evidence collection, SBOM generation, and artifact integrity validation.

4. Line-by-Line Security Explanation

Dockerfile Area	Explanation
# syntax=docker/dockerfile:1.7	Uses a modern Dockerfile frontend syntax and supports cleaner, more predictable Docker BuildKit behavior.
ARG PYTHON_VERSION=3.11.9	Keeps the Python version explicit and reproducible. Version pinning helps avoid uncontrolled base image drift.
builder stage	Creates a dedicated stage for dependency installation. This keeps dependency installation behavior separate from the final runtime layer.
ENV PYTHONDONTWRITEBYTECODE=1	Prevents Python from writing .pyc files, reducing unnecessary filesystem writes and layer noise.
ENV PYTHONUNBUFFERED=1	Improves log visibility by sending Python output directly to stdout/stderr, useful for Docker and Kubernetes logs.
PIP_NO_CACHE_DIR=1	Prevents pip cache from increasing image size and leaving unnecessary files in image layers.
python -m venv /opt/venv	Creates an isolated runtime dependency environment that can be copied into the final image.
runtime stage	Creates a clean final image containing only the runtime components required to start the app.
OCI LABEL fields	Adds metadata used by security reviewers, SBOM tools, registries, and artifact documentation.
addgroup/adduser fixed UID	Creates a deterministic non-root identity that can be referenced in Kubernetes SecurityContext.
COPY --chown	Ensures app files are owned by the application user instead of root.
chmod 0444 and 0555	Makes application files and directories read-only where possible.
chmod 1777 /tmp/app	Provides a controlled temporary write area with sticky-bit permissions.
USER 10001:10001	Ensures runtime execution does not happen as root. This is one of the most important container hardening controls.
HEALTHCHECK	Provides container-level availability signal. It supports operational monitoring and failure detection.
Gunicorn CMD	Starts the app with a production-style WSGI server and logs to stdout/stderr for observability.

5. Security Controls Implemented

5.1 Build-Time Controls

- Dependencies are installed in a dedicated builder stage.
- Pip cache is disabled to reduce image size and prevent unnecessary files in layers.
- Build behavior is controlled through pinned Python base version.
- The runtime stage is separated from build-time operations.
- Only required application files and virtual environment are copied to runtime.

5.2 Runtime Controls

- The application process runs as non-root user 10001.
- The container exposes only application port 5000.
- The Flask development server is not used as the default runtime process.
- Application files are made read-only to reduce tampering risk.
- Temporary writes are directed toward /tmp/app where possible.
- The container includes a HEALTHCHECK for operational validation.

5.3 Supply Chain Controls

- OCI image labels improve metadata traceability.
- The image can be scanned with Trivy for vulnerabilities and misconfigurations.
- The image can be inventoried with Syft to produce SPDX and CycloneDX SBOMs.
- The image can be signed and verified using Cosign.
- Image digest can be recorded in supply chain evidence reports.

6. Build and Runtime Commands

Use the following commands to build, run, and validate the hardened Dockerfile in the private VM environment.

6.1 Build Hardened Image

```
docker build -f Dockerfile.hardened -t devsecops-vuln-app:hardened .
```

6.2 Run Hardened Image Locally

```
docker run --rm -p 127.0.0.1:8080:5000 --name devsecops-hardened devsecops-vuln-app:hardened
```

6.3 Verify Health Endpoint

```
curl -fsS http://127.0.0.1:8080/health
```

6.4 Verify Non-Root Runtime

```
docker run --rm --entrypoint id devsecops-vuln-app:hardened
```

Expected evidence should show uid=10001 and gid=10001. This proves the image is not running as root.

7. Security Scanning and Evidence Commands

The hardened Dockerfile should be validated through scanner output and saved under security/reports. These commands help generate evidence that can be added to the evidence index and professional reports.

```
mkdir -p security/reports security/sbom

# Dockerfile and repository scan
trivy fs --scanners vuln,secret,misconfig \
  --severity CRITICAL,HIGH,MEDIUM \
  --format table . | tee security/reports/trivy-fs-hardened-report.txt

# Hardened image vulnerability scan
trivy image --severity CRITICAL,HIGH,MEDIUM \
  --ignore-unfixed \
  --format table devsecops-vuln-app:hardened \
  | tee security/reports/trivy-image-hardened-report.txt

# SBOM generation with Syft
syft devsecops-vuln-app:hardened -o spdx-json=security/sbom/hardened-sbom-spdx.json
syft devsecops-vuln-app:hardened -o cyclonedx-json=security/sbom/hardened-sbom-cyclonedx.json

# Package count evidence
jq '.packages | length' security/sbom/hardened-sbom-spdx.json
```

```
# Optional image digest evidence after pushing image
# docker buildx imagetools inspect
ghcr.io/YOUR_GITHUB_USERNAME/enterprise-devsecops-lab:hardened
```

8. Kubernetes Compatibility

The hardened image is designed to work with Kubernetes security controls in the k8s/base deployment. The fixed non-root UID/GID, minimal runtime process, and application health endpoint support a stronger Kubernetes deployment posture.

Kubernetes Control	Dockerfile Support	Expected Validation
runAsNonRoot	Container runs as USER 10001:10001.	Pod starts without needing root privileges.
runAsUser/runAsGroup	Fixed UID/GID allow deterministic SecurityContext.	kubect! get pod jsonpath shows UID/GID.
allowPrivilegeEscalation: false	Application does not require privileged behavior.	Pod remains functional with escalation disabled.
capabilities.drop: ALL	Application does not require Linux capabilities.	App health endpoint remains available.
readOnlyRootFilesystem	App files are read-only; /tmp/app can be mounted as writable if needed.	Writes to /app fail; writes to /tmp/app pass if mounted.
seccomp RuntimeDefault	Application uses normal userspace behavior.	Pod starts under RuntimeDefault seccomp profile.
liveness/readiness probes	HEALTHCHECK and /health route support probe design.	Kubernetes probes report healthy state.

9. CI/CD Security Gate Integration

Dockerfile.hardened should be integrated into GitHub Actions security gates so the image is built, scanned, and rejected if critical issues are discovered. This helps convert the Dockerfile from a static artifact into a controlled security checkpoint.

name: Hardened Dockerfile Security Check

```
on:
  pull_request:
    branches: [main]
  workflow_dispatch:

jobs:
  hardened-image-scan:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Build hardened image
        run: docker build -f Dockerfile.hardened -t devsecops-vuln-app:hardened .

      - name: Scan hardened image with Trivy
        uses: aquasecurity/trivy-action@v0.36.0
        with:
          image-ref: devsecops-vuln-app:hardened
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          format: sarif
          output: trivy-hardened-image.sarif
          exit-code: "1"

      - name: Upload SARIF
        if: always()
        uses: github/codeql-action/upload-sarif@v4
        with:
          sarif_file: trivy-hardened-image.sarif
```

10. Risk Considerations and Limitations

Container hardening reduces the blast radius of application compromise, but it does not automatically fix vulnerable code. The vulnerable Flask application intentionally contains AppSec issues for lab demonstration. These must still be remediated or documented as accepted lab risk.

Risk / Limitation	Why It Matters	Recommended Action
Vulnerable application routes remain	SQL injection, command injection, and unsafe rendering are application-layer issues.	Fix code or keep documented as intentional lab findings.
Dependency vulnerabilities may remain	Base image and Python packages may still contain vulnerabilities.	Run Trivy and pip-audit; document fixed and accepted risk.
SQLite write location may break read-only runtime	If the app writes to /app, read-only filesystem will block writes.	Move runtime writes to /tmp/app or mount a Kubernetes emptyDir.
Slim image is not distroless	Slim reduces footprint but still has OS packages.	Use Trivy results to decide whether distroless is worth future improvement.
Cosign key handling risk	Private signing keys must not be committed.	Keep cosign.key ignored and use GitHub secrets for CI/CD signing.
Healthcheck does not prove security	It only proves application availability.	Pair with tests, scans, runtime detection, and policy validation.

11. Evidence Checklist

Capture screenshots and save reports to make the Dockerfile hardening visible to recruiters, reviewers, and auditors. Recommended evidence names are listed below.

Evidence Item	Command / Source	Suggested Filename
Hardened image build	<code>docker build -f Dockerfile.hardened -t devsecops-vuln-app:hardened .</code>	dockerfile-hardened-build-success.png
Non-root user proof	<code>docker run --rm --entrypoint id devsecops-vuln-app:hardened</code>	dockerfile-hardened-nonroot-proof.png
Healthcheck proof	<code>curl http://127.0.0.1:8080/health</code>	dockerfile-hardened-healthcheck-proof.png
Trivy image scan	<code>trivy image devsecops-vuln-app:hardened</code>	dockerfile-hardened-trivy-image-scan.png
Trivy filesystem scan	<code>trivy fs .</code>	dockerfile-hardened-trivy-fs-scan.png
Syft SBOM	<code>syft devsecops-vuln-app:hardened</code>	dockerfile-hardened-syft-sbom.png
SBOM package count	<code>jq .packages length</code>	dockerfile-hardened-sbom-package-count.png
Kubernetes SecurityContext compatibility	<code>kubectl get pod jsonpath securityContext</code>	dockerfile-hardened-k8s-securitycontext.png

12. Control Validation Matrix

Control	Validation Method	Pass Criteria
Build succeeds	<code>docker build -f Dockerfile.hardened</code>	Image builds without error.
Non-root execution	<code>docker run --entrypoint id</code>	UID/GID are 10001, not 0.
Application availability	<code>curl /health</code>	Endpoint returns healthy response.
No critical image vulnerability	<code>trivy image</code>	No unresolved critical vulnerabilities unless documented as exception.
No secret in build context	<code>trivy fs --scanners secret</code>	No real secrets detected.

Control	Validation Method	Pass Criteria
SBOM generated	syft spdx-json and cyclonedx-json	SBOM files exist and contain packages/components.
Kubernetes compatibility	kubectl apply -k k8s/base	Pod reaches Running/Ready state.
Read-only runtime compatibility	Kubernetes readOnlyRootFilesystem or Docker runtime tests	App works without needing writes to /app.

13. Compliance and Framework Alignment

This Dockerfile.hardened artifact supports multiple framework-aligned security practices. The alignment below is for portfolio and audit-style documentation only; it does not claim formal certification.

Framework / Standard	Relevant Alignment
NIST SSDF	Supports secure build practices, vulnerability detection, and artifact traceability through scanning and evidence collection.
OWASP ASVS	Supports secure deployment and configuration practices around runtime behavior, error handling, and security verification.
CIS Docker style controls	Aligns with non-root containers, reduced privileges, image scanning, and controlled runtime configuration.
CIS Kubernetes Benchmark	Supports workload security expectations such as non-root execution, least privilege, and restricted runtime configuration.
SLSA	Supports build integrity and artifact traceability when paired with provenance and CI/CD evidence.
CycloneDX / SPDX	Supports software component transparency through SBOM generation.
SOC 2 style controls	Supports change management, vulnerability management, monitoring, and evidence retention.
ISO 27001 style controls	Supports secure development, vulnerability handling, access control, and operational security evidence.

14. Recommended Repository Placement

Place the hardened Dockerfile in the repository root and reference it in documentation and security gates.

```
Enterprise-DevSecOps/  
├── Dockerfile  
├── Dockerfile.hardened  
├── docker-compose.yml  
├── src/app/  
├── tests/  
├── k8s/base/  
├── security/  
│   ├── reports/  
│   ├── sbom/  
│   └── provenance/  
├── docs/  
│   └── reports/  
└── README.md
```

The original Dockerfile can remain for comparison or lab baseline. Dockerfile.hardened should be used as the secure target artifact for evidence, remediation, and CI/CD hardening.

15. Retest Proof Template

Use the following table when adding Dockerfile hardening evidence to the Retest Proof document.

Retest Area	Expected Result	Evidence Path
Build validation	Hardened image builds successfully.	evidence/screenshots/dockerfile-hardened-build-success.png

Retest Area	Expected Result	Evidence Path
Non-root validation	Container runs as UID/GID 10001.	evidence/screenshots/dockerfile-hardened-nonroot-proof.png
Health validation	/health endpoint responds successfully.	evidence/screenshots/dockerfile-hardened-healthcheck-proof.png
Trivy image scan	Critical findings remediated or documented as exception.	security/reports/trivy-image-hardened-report.txt
SBOM validation	SPDX and CycloneDX SBOMs generated.	security/sbom/hardened-sbom-spdx.json
Kubernetes validation	Pod runs with hardened SecurityContext.	evidence/screenshots/dockerfile-hardened-k8s-securitycontext.png

16. Final Recommendation

Dockerfile.hardened should be added as a secure target artifact in the Enterprise DevSecOps project. It demonstrates that the project understands container hardening beyond scanner output. The document should be committed with Dockerfile.hardened, linked from README.md, referenced in security-gates.yml, and supported by screenshots and reports in the evidence index.

For maximum professional value, keep both the baseline Dockerfile and Dockerfile.hardened. The baseline shows the starting point, while Dockerfile.hardened shows remediation maturity. This contrast makes the project stronger in interviews because it demonstrates before-and-after security engineering.